Task#1

```
#include <stdio.h>


int main() {
   int matrix[3][3]={ {1, 23, 3}, {4, 63, 6}, {7, 33, 9} }; // i take these sample because of every time checks
   int i, j, k, l;


   // Input the 3x3 matrix
   // printf("Enter the elements of the 3x3 matrix\n");
   // for (i = 0; i < 3; i++) {
   //     for (j = 0; j < 3; j++) {
   //         scanf("%f", &matrix[i][j]);
   //     }
   //     printf("\n");
   // }


   //Original matrix
   printf("\nOriginal Matrix:\n");
   for (i = 0; i < 3; i++) {
      for (j = 0; j < 3; j++) {
         printf("%d ", matrix[i][j]);
      }
      printf("\n");
   }


   //Transpose Matrix
   int trans[3][3];
   for (i = 0; i < 3; i++) {
```

```
    for (j = 0; j < 3; j++) {
        trans[j][i] = matrix[i][j];
    }
}
printf("\nTranspose of the Matrix:\n");
for (i = 0; i < 3; i++) {
    for (j = 0; j < 3; j++) {
        printf("%d ", trans[i][j]);
    }
    printf("\n");
}

//determinant
float det = matrix[0][0] * (matrix[1][1] * matrix[2][2] - matrix[1][2] * matrix[2][1])
          - matrix[0][1] * (matrix[1][0] * matrix[2][2] - matrix[1][2] * matrix[2][0])
          + matrix[0][2] * (matrix[1][0] * matrix[2][1] - matrix[1][1] * matrix[2][0]);
printf("\nDeterminant of the Matrix: %.2f\n", det);

//  cofactor matrix
float cof[3][3];
for (i = 0; i < 3; i++) {
    for (j = 0; j < 3; j++) {
        // minor matrix (2x2)
        float minor_mat[2][2];
        int row = 0;
        for (k = 0; k < 3; k++) {
            if (k != i) {
                int col = 0;
                for (l = 0; l < 3; l++) {
```

```
                if (l != j) {
                    minor_mat[row][col] = matrix[k][l];
                    col++;
                }
            }
            row++;
        }
    }
    // Determinant of minor
    float det_minor = minor_mat[0][0] * minor_mat[1][1] - minor_mat[0][1] * minor_mat[1][0];
    // Cofactor
    cof[i][j] = ((i + j) % 2 == 0 ? 1 : -1) * det_minor;
  }
}
printf("\nCofactor Matrix:\n");
for (i = 0; i < 3; i++) {
  for (j = 0; j < 3; j++) {
    printf("%.2f ", cof[i][j]);
  }
  printf("\n");
}

// adjoint matrix
float adj[3][3];
for (i = 0; i < 3; i++) {
  for (j = 0; j < 3; j++) {
    adj[j][i] = cof[i][j];
  }
}
```

```
    printf("\nAdjoint Matrix:\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            printf("%.2f ", adj[i][j]);
        }
        printf("\n");
    }

    // inverse if determinant is not zero
    if (det != 0) {
        float inv[3][3];
        for (i = 0; i < 3; i++) {
            for (j = 0; j < 3; j++) {
                inv[i][j] = adj[i][j] / det;
            }
        }
        printf("\nInverse Matrix:\n");
        for (i = 0; i < 3; i++) {
            for (j = 0; j < 3; j++) {
                printf("%.2f ", inv[i][j]);
            }
            printf("\n");
        }
    } else {
        printf("\nThe matrix is not invertible because determinant is zero.\n");
    }

    return 0;
}
```

Task#2

```
#include <stdio.h>

int main() {
    int rows, cols, i, j, k, l;
    float matrix[5][5], matrix2[5][5];  // For two matrices if needed
    int isSquare = 0, isRect = 0, isZero = 1, isIdentity = 1, isDiagonal = 1, isScalar = 1;
    int isUpperTri = 1, isLowerTri = 1, isSymmetric = 1, isSkewSym = 1, isSingular = 0, isEqual = 1;
    int isRow = 0, isColumn = 0, isNull = 1, isIdempotent = 1, isNilpotent = 1;
    float determinant = 0, value = 0;

    // Read dimensions
    printf("Enter rows and columns (up to 5): ");
    scanf("%d %d", &rows, &cols);

    // Read first matrix
    printf("Enter elements of first matrix:\n");
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            scanf("%f", &matrix[i][j]);
        }
    }

    // Read second matrix for equality check
    printf("Enter elements of second matrix for equality check:\n");
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            scanf("%f", &matrix2[i][j]);
```

```
    }
}

// Check basic types
if (rows == cols) {
    isSquare = 1;
    isRect = 0;
} else {
    isSquare = 0;
    isRect = 1;
}

if (rows == 1) isRow = 1;
if (cols == 1) isColumn = 1;

// Check zero/null
for (i = 0; i < rows; i++) {
    for (j = 0; j < cols; j++) {
        if (matrix[i][j] != 0) {
            isZero = 0;
            isNull = 0;
        }
    }
}

// Check equality
for (i = 0; i < rows; i++) {
    for (j = 0; j < cols; j++) {
        if (matrix[i][j] != matrix2[i][j]) {
```

```
            isEqual = 0;
        }
    }
}

// For square matrices only
if (isSquare) {
    // Get value for scalar (first diagonal)
    value = matrix[0][0];

    // Check identity, diagonal, scalar, triangular, symmetric, skew-symmetric
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            if (i == j) {
                if (matrix[i][j] != 1) isIdentity = 0;
                if (matrix[i][j] != value) isScalar = 0;
                if (matrix[i][j] != 0) isSkewSym = 0;  // Diagonal must be 0 for skew-symmetric
            } else {
                if (matrix[i][j] != 0) {
                    isIdentity = 0;
                    isDiagonal = 0;
                    isScalar = 0;
                }
                if (i < j && matrix[i][j] != 0) isLowerTri = 0;
                if (i > j && matrix[i][j] != 0) isUpperTri = 0;
            }
            if (matrix[i][j] != matrix[j][i]) isSymmetric = 0;
            if (matrix[i][j] != -matrix[j][i]) isSkewSym = 0;
        }
```

```
}

// Determinant for singular/non-singular
if (rows == 2) {
   determinant = matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0];
} else if (rows == 3) {
   determinant = matrix[0][0] * (matrix[1][1] * matrix[2][2] - matrix[1][2] * matrix[2][1])
      - matrix[0][1] * (matrix[1][0] * matrix[2][2] - matrix[1][2] * matrix[2][0])
      + matrix[0][2] * (matrix[1][0] * matrix[2][1] - matrix[1][1] * matrix[2][0]);
}
if (determinant == 0) isSingular = 1;  // Singular if determinant == 0

// Idempotent: A * A == A
float prod[5][5] = {0};
for (i = 0; i < rows; i++) {
   for (j = 0; j < cols; j++) {
      for (k = 0; k < cols; k++) {
         prod[i][j] += matrix[i][k] * matrix[k][j];
      }
      if (prod[i][j] != matrix[i][j]) isIdempotent = 0;
   }
}

// Nilpotent: A^2 == 0
for (i = 0; i < rows; i++) {
   for (j = 0; j < cols; j++) {
      if (prod[i][j] != 0) isNilpotent = 0;
   }
}
```

```
    }

    // Display results
    printf("Matrix types:\n");
    if (isSquare) printf("Square Matrix\n");
    if (isRect) printf("Rectangular Matrix\n");
    if (isZero) printf("Zero Matrix\n");
    if (isIdentity) printf("Identity Matrix\n");
    if (isDiagonal) printf("Diagonal Matrix\n");
    if (isScalar) printf("Scalar Matrix\n");
    if (isUpperTri) printf("Upper Triangular Matrix\n");
    if (isLowerTri) printf("Lower Triangular Matrix\n");
    if (isSymmetric) printf("Symmetric Matrix\n");
    if (isSkewSym) printf("Skew-Symmetric Matrix\n");
    if (isSingular) printf(" Singular\n"); else if (isSquare) printf("11. Non-Singular\n");
    if (isEqual) printf(" Equal Matrix\n");
    if (isRow) printf(" Row Matrix\n");
    if (isColumn) printf(" Column Matrix\n");
    if (isNull) printf(" Null Matrix\n");
    if (isIdempotent) printf("16. Idempotent Matrix\n");
    if (isNilpotent) printf("17. Nilpotent Matrix\n");

    return 0;
}
```

Task#3

```
#include <stdio.h>

int main() {
   int rowsA, colsA, rowsB, colsB, i, j, k;
   float matrixA[3][3], matrixB[3][3], result[3][3];



   printf("Enter rows and columns for matrix A (up to 3x3): ");
   scanf("%d %d", &rowsA, &colsA);

   printf("Enter rows and columns for matrix B (up to 3x3): ");
   scanf("%d %d", &rowsB, &colsB);

   // check cols and rows for multiplication condition
   if (colsA != rowsB) {
      printf("Columns of matrix A must equal rows of matrix B for multiplication.\n");
      return 1;
   }

   // matrix A
   printf("Enter elements of matrix A:\n");
   for (i = 0; i < rowsA; i++) {
      for (j = 0; j < colsA; j++) {
         scanf("%f", &matrixA[i][j]);
      }
   }
```

```
//  matrix B
printf("Enter elements of matrix B:\n");
for (i = 0; i < rowsB; i++) {
   for (j = 0; j < colsB; j++) {
      scanf("%f", &matrixB[i][j]);
   }
}

// zero matrix
for (i = 0; i < rowsA; i++) {
   for (j = 0; j < colsB; j++) {
      result[i][j] = 0;
   }
}

// Multiplication of two matrices
for (i = 0; i < rowsA; i++) {
   for (j = 0; j < colsB; j++) {
      for (k = 0; k < colsA; k++) {
         result[i][j] += matrixA[i][k] * matrixB[k][j];
      }
   }
}

// Print matrix A
printf("\nMatrix A:\n");
```

```
    for (i = 0; i < rowsA; i++) {
        for (j = 0; j < colsA; j++) {
            printf("%.2f ", matrixA[i][j]);
        }
        printf("\n");
    }

    // Print matrix B
    printf("\nMatrix B:\n");
    for (i = 0; i < rowsB; i++) {
        for (j = 0; j < colsB; j++) {
            printf("%.2f ", matrixB[i][j]);
        }
        printf("\n");
    }
    // Print result matrix
    printf("\nResult Matrix (A * B):\n");
    for (i = 0; i < rowsA; i++) {
        for (j = 0; j < colsB; j++) {
            printf("%.2f ", result[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Task#4

```
#include <stdio.h>

int main() {
   int array[3][3][3];
   int i, j, k;
   int layerTotal[3] = {0};
   int overallTotal = 0, overallMax = 0, overallMin, count = 0;
   float overallAverage;

   // input elements for the 3D array
   printf("Enter 27 elements for the 3x3x3 array (layer by layer):\n");
   for (i = 0; i < 3; i++) {
      printf("Layer %d:\n", i + 1);
      for (j = 0; j < 3; j++) {
         for (k = 0; k < 3; k++) {
            scanf("%d", &array[i][j][k]);
         }
      }
   }

   // Traverse and check layer-wise totals and overall result
   for (i = 0; i < 3; i++) {
      printf("\nLayer %d:\n", i + 1);
      for (j = 0; j < 3; j++) {
         for (k = 0; k < 3; k++) {
            printf("%d ", array[i][j][k]);
```

```
            layerTotal[i] += array[i][j][k];
            overallTotal += array[i][j][k];
            if (array[i][j][k] > overallMax) overallMax = array[i][j][k];
            if (array[i][j][k] < overallMin) overallMin = array[i][j][k];
            count++;
        }
        printf("\n");
    }
    printf("Total for Layer %d: %d\n", i + 1, layerTotal[i]);
}

// Overall result
overallAverage = (float)overallTotal / count;
printf("\nOverall Result:\n");
printf("Total Sum: %d\n", overallTotal);
printf("Average: %.2f\n", overallAverage);
printf("Maximum: %d\n", overallMax);
printf("Minimum: %d\n", overallMin);

return 0;
}
```

Task#5

```
#include <stdio.h>


int main() {
   int array[3][3][3];  // 3x3x3 3D array
   int i, j, k, layer1, layer2;
   int isIdentical;

   // Read elements for the 3D array
   printf("Enter 27 elements for the 3x3x3 array (layer by layer):\n");
   for (i = 0; i < 3; i++) {
      printf("Layer %d:\n", i + 1);
      for (j = 0; j < 3; j++) {
         for (k = 0; k < 3; k++) {
            scanf("%d", &array[i][j][k]);
         }
      }
   }

   // Compare each pair of layers
   printf("\nLayer Similarity Report:\n");
   for (layer1 = 0; layer1 < 3; layer1++) {
      for (layer2 = layer1 + 1; layer2 < 3; layer2++) {
         isIdentical = 1;  // Assume identical
         for (j = 0; j < 3; j++) {
            for (k = 0; k < 3; k++) {
               if (array[layer1][j][k] != array[layer2][j][k]) {
```

```
                isIdentical = 0;
                break;
            }
        }
        if (!isIdentical) break;
    }
    printf("Layer %d and Layer %d: ", layer1 + 1, layer2 + 1);
    if (isIdentical) {
        printf("Identical\n");
    } else {
        printf("Distinct\n");
    }
  }
}

  return 0;
}
```